

TCP/IP Chat and Wireshark Traffic Analysis

Python sockets • TCP/IP • Wireshark • Jupyter Notebook

Short summary: This project demonstrates how application data is sent over a TCP/IP network. It includes a small HTTP traffic simulation in Jupyter Notebook and a real TCP chat application built with Python sockets. The project also includes Wireshark captures to show how the traffic looks at the packet level.

Part	What it shows	Main tools
Part 1	HTTP messages are loaded from a CSV file, wrapped into TCP/IP layers, and captured in Wireshark.	CSV, Jupyter Notebook, Wireshark
Part 2	A real client-server chat application sends messages between users through a TCP server.	Python, socket, threading, Wireshark

1. Project Goal

The goal of this project is to connect theory with real network behavior. Instead of only learning that HTTP runs over TCP and TCP runs over IP, the project shows this process using real code and Wireshark captures. The main idea is simple: an application message, such as an HTTP request or a chat message, does not travel alone. It is carried inside TCP segments, which are carried inside IP packets.

- Understand how application-layer data is encapsulated inside TCP and IP.
- Build a simple TCP client-server chat application using Python.
- Use threads so that multiple clients can communicate at the same time.
- Capture and explain the traffic using Wireshark.
- Recognize important TCP fields such as SYN, ACK, Seq, Ack, Len and Window Size.

2. Project Files

File	Purpose
server.py	Runs the TCP chat server. It listens for clients and routes messages between them.
client.py	Runs a chat client. It connects to the server, sends a username, sends messages and receives messages.
data_input_main.csv	Contains example HTTP application-layer messages for Part 1.
traffic_demo_http.ipynb	Loads the CSV, simulates TCP/IP encapsulation and creates HTTP traffic for Wireshark.

3. Part 1 - HTTP Traffic Analysis

3.1 CSV Input

The CSV file represents application-layer messages. Each row describes one message: who sent it, who received it, which application protocol was used, what the message content was, and at what time it happened.

```
msg_id,app_protocol,src_app,dst_app,message,timestamp
1,HTTP,client_browser,web_server,GET /index.html,0.015
2,HTTP,web_server,client_browser,HTTP/1.1 200 OK,0.032
```

Column	Meaning in simple words
msg_id	A unique number for the message.
app_protocol	The application protocol. In this project, Part 1 uses HTTP.
src_app	The application that sends the message, for example client_browser.
dst_app	The application that receives the message, for example web_server.
message	The actual application message, such as GET /index.html.
timestamp	The time of the message in the scenario.

Human explanation: the CSV is not a packet capture by itself. It is a structured input file that describes the messages we want to simulate and analyze.

3.2 Jupyter Step 2 - TCP/IP Encapsulation Simulation

In this step, the notebook takes the application message and adds the network information that would exist in real traffic. The HTTP message is treated as application data. TCP adds source and destination ports. IP adds source and destination IP addresses.

```
Application Layer: GET /index.html
TCP Layer: Source Port = 51515, Destination Port = 8080
IP Layer: Source IP = 127.0.0.1, Destination IP = 127.0.0.1
```

A good way to explain it: the data is wrapped like envelopes. The HTTP message is inside TCP, and TCP is inside IP.

```
[ IP [ TCP [ HTTP DATA ] ] ]
```

Field	Why it exists
Source IP	Shows which machine sent the packet.
Destination IP	Shows which machine should receive the packet.
Source Port	The port used by the sender. For a client, this is usually a temporary port.
Destination Port	The port used by the server application, such as 8080 for the HTTP demo.

3.3 Jupyter Step 3 - Creating Real HTTP Traffic

Step 3 creates real HTTP traffic so Wireshark can capture it. In the notebook, the HTTP demo runs locally on the same computer.

```
HTTP_HOST = "127.0.0.1"
HTTP_PORT = 8080
```

Value	Meaning
127.0.0.1	Localhost / loopback. The computer communicates with itself.
8080	The port used by the temporary HTTP server in the notebook.
lo0 / Loopback	The Wireshark interface used when traffic stays inside the same computer.

Important: because the HTTP demo uses 127.0.0.1, the correct Wireshark interface is Loopback / lo0, not Wi-Fi.

3.4 Wireshark Capture for the HTTP Demo

1. Open Wireshark before running the Jupyter traffic cell.
2. Choose the Loopback / lo0 interface because the traffic uses 127.0.0.1.
3. Start the capture.
4. Use the display filter: `tcp.port == 8080`.
5. Run Step 3 in the Jupyter Notebook.
6. Look for HTTP requests such as `GET /index.html` and responses such as `HTTP/1.0 200 OK`.
7. Stop the capture and save the file if required.

```
tcp.port == 8080
```

What this proves: Wireshark shows that the notebook created real HTTP traffic. First TCP opens a connection, then the HTTP request is sent, then the server returns a successful response.

4. Part 2 - TCP Chat Application

4.1 Architecture

The chat application uses a simple client-server architecture. The server is the central point. Clients do not send messages directly to each other. Instead, each client sends its message to the server, and the server forwards it to the target client.

```
Alice client -> Server -> Bob client
```

Component	Role
Server	Listens for clients, stores connected users and forwards messages.
Client	Connects to the server, sends a username, sends messages and receives incoming messages.
Socket	The communication endpoint used by the program to send and receive network data.
Thread	Allows the program to do more than one task at the same time.

4.2 Server Workflow

8. The server creates a TCP socket.
9. It binds the socket to an IP address and port, for example 0.0.0.0:50001.
10. It starts listening for incoming clients.
11. When a client connects, the server accepts the connection.
12. The server receives the client username and stores it in a dictionary.
13. For every connected client, the server runs a separate thread.
14. When a message arrives, the server reads the target name and forwards the message to the correct socket.

```
clients[name] = client_socket
# This means: username -> socket connection
```

Human explanation: the dictionary works like a contact list. If the user types Bob:Hello, the server looks for Bob in the dictionary and sends the message through Bob's socket.

4.3 Client Workflow

15. The client creates a TCP socket.
16. The client connects to the server IP address and port.
17. The client sends a unique username to the server.
18. A background thread starts listening for incoming messages from the server.
19. The main part of the program keeps reading user input from the keyboard.
20. If the user types TargetName:Message, the client sends it to the server.
21. If the user types quit, the client closes the socket and exits.

```
TargetName:Message
Bob:Hello Bob
```

Why a receive thread is needed: without a separate thread, the client would either wait for keyboard input or wait for server messages, but not both at the same time. The thread allows the client to receive messages while the user is still typing.

5. How to Run the Chat Application

5.1 Run the Server

```
python3 server.py
```

Recommended server configuration:

```
HOST = "0.0.0.0"
PORT = 50001
```

0.0.0.0 means the server listens on all network interfaces of the server computer. It is used on the server side only.

5.2 Run Clients

```
python3 client.py
```

In client.py, replace SERVER_IP_HERE with the actual local IP address of the server computer. The port must be the same port used by the server.

```
HOST = "SERVER_IP_HERE"
PORT = 50001
```

To test five clients: open five terminal windows and run client.py in each one. Use different usernames such as Alice, Bob, Charlie, David and Eve.

6. Wireshark Capture for the Chat Application

For the chat application, use the Wi-Fi interface when the server and client are on different computers. Use Loopback only when both sides run on the same computer using 127.0.0.1.

Scenario	Wireshark interface	Display filter
Jupyter HTTP demo on one computer	Loopback / lo0	tcp.port == 8080
Chat app between two computers on Wi-Fi	Wi-Fi / en0	tcp.port == 50001

```
tcp.port == 50001
```

In the chat capture, the server port is fixed: 50001. Client ports are temporary ports assigned automatically by the operating system, for example 58971 or 58975.

```
Client:58971 -> Server:50001 [SYN]
Server:50001 -> Client:58971 [SYN, ACK]
Client:58971 -> Server:50001 [ACK]
```

This is the **TCP three-way handshake**. After these three packets, the TCP connection is established and real application data can be sent.

7. Important Wireshark and TCP/IP Terms

Term	Simple explanation
SYN	A TCP flag used to start a connection. The client sends SYN to the server.
SYN, ACK	The server accepts the connection request and acknowledges it.
ACK	Acknowledgment. It means a packet or data was received.
FIN, ACK	A normal TCP connection close.
RST, ACK	A sudden or immediate connection reset.
PSH, ACK	Usually means real application data is being pushed, together with an acknowledgment.
Seq	Sequence number. TCP uses it to keep the data in the correct order.
Ack	Acknowledgment number. It shows what data was received and what is expected next.
Len	The amount of TCP payload data in this segment. Len=0 is usually control traffic; Len>0 means real data.
Win	TCP window size. It shows how much data the receiver can accept.
MSS	Maximum Segment Size. The largest TCP data segment size the side is willing to receive.
Source IP	The IP address of the machine that sent the packet.
Destination IP	The IP address of the machine that should receive the packet.
Source Port	The port used by the sender. For clients, usually a temporary port.
Destination Port	The port used by the receiver application, such as 50001 on the chat server.

8. Common Problems and Fixes

Problem	Likely reason	Fix
No packets in Wireshark for Jupyter	Wrong interface selected.	Use Loopback / lo0 and filter tcp.port == 8080.
No packets in Wireshark for chat	Wrong interface selected or wrong port.	Use Wi-Fi / en0 and filter tcp.port == 50001.
Client cannot connect	Client is using the wrong server IP or firewall blocks Python.	Use the server computer IP and allow Python through firewall.
Message is not delivered	Target username does not match exactly.	Use the exact username, including capital letters.
Terminal looks stuck after receiving a message	Incoming message was printed while input() was waiting.	Keep typing after the > prompt and press Enter.

9. What I Learned

This project helped me understand that network communication is not just code that sends text. Every message becomes TCP data, and every TCP segment is carried inside IP packets. Wireshark made it possible to see the process clearly: connection opening, data transfer, acknowledgments, retransmissions and connection closing.

- A socket is the bridge between the application and the network.
- TCP is connection-oriented and uses a three-way handshake before data transfer.
- Threads are useful when a program must handle receiving and sending at the same time.
- Wireshark helps connect code behavior to real packet behavior.
- A server port is fixed, while client ports are usually temporary.
- Len>0 in TCP usually indicates that real application data is being transferred.

10. Quick Presentation Answers

What is a socket? A socket is a communication endpoint that allows an application to send and receive data over the network.

Why did you use TCP? TCP is reliable, connection-oriented and keeps data in order, which fits a chat application.

What is a thread? A thread is a separate flow of execution. It allows the client to receive messages while the user is typing, and allows the server to handle multiple clients.

How do you see the TCP handshake? In Wireshark it appears as SYN, then SYN-ACK, then ACK.

How do you know real data passed? Look for TCP packets with PSH, ACK and Len greater than 0.

Why are there two ports? Every TCP packet has a source port and a destination port. The server has a fixed port and the client usually has a temporary port.

Why does Wireshark show TCP and not CHAT? The chat protocol is custom application data over TCP. Wireshark recognizes TCP but does not know the custom chat format.